



DATA STRUCTURES & ALGORITHMS *in* PYTHON

Rough Cuts



JOHN CANNING | ALAN BRODER
ROBERT LAFORE

Data Structures & Algorithms in Python

Data Structures & Algorithms in Python

**John Canning
Alan Broder
Robert Lafore**

 **Addison-Wesley**

**Boston • Columbus • New York • San Francisco • Amsterdam • Cape Town
Dubai • London • Madrid • Milan • Munich • Paris • Montreal • Toronto • Delhi • Mexico City
São Paulo • Sidney • Hong Kong • Seoul • Singapore • Taipei • Tokyo**

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed with initial capital letters or in all capitals.

The authors and publisher have taken care in the preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

For information about buying this title in bulk quantities, or for special sales opportunities (which may include electronic versions; custom cover designs; and content particular to your business, training goals, marketing focus, or branding interests), please contact our corporate sales department at corpsales@pearsoned.com or (800) 382-3419.

For government sales inquiries, please contact governmentsales@pearsoned.com.

For questions about sales outside the U.S., please contact intlcs@pearson.com.

Visit us on the Web: informit.com/aw

Library of Congress Control Number: 2022910068

Copyright © 2023 Pearson Education, Inc.

All rights reserved. Printed in the United States of America. This publication is protected by copyright, and permission must be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. For information regarding permissions, request forms and the appropriate contacts within the Pearson Education Global Rights & Permissions Department, please visit www.pearsoned.com/permissions/.

ISBN-13: 978-0-13-485568-4

ISBN-10: 0-13-485568-X

ScoutAutomatedPrintCode

Editor-in-Chief

Mark Taub

Director, ITP Product Management

Brett Bartow

Acquisitions Editor

Kim Spenceley

Development Editor

Chris Zahn

Managing Editor

Sandra Schroeder

Project Editor

Mandie Frank

Copy Editor

Chuck Hutchinson

Indexer

Proofreader

Editorial Assistant

Cindy Teeters

Designer

Chuti Prasertsith

Compositor

codeMantra

Pearson's Commitment to Diversity, Equity, and Inclusion

Pearson is dedicated to creating bias-free content that reflects the diversity of all learners. We embrace the many dimensions of diversity, including but not limited to race, ethnicity, gender, socioeconomic status, ability, age, sexual orientation, and religious or political beliefs.

Education is a powerful force for equity and change in our world. It has the potential to deliver opportunities that improve lives and enable economic mobility. As we work with authors to create content for every product and service, we acknowledge our responsibility to demonstrate inclusivity and incorporate diverse scholarship so that everyone can achieve their potential through learning. As the world's leading learning company, we have a duty to help drive change and live up to our purpose to help more people create a better life for themselves and to create a better world.

Our ambition is to purposefully contribute to a world where

- Everyone has an equitable and lifelong opportunity to succeed through learning
- Our educational products and services are inclusive and represent the rich diversity of learners
- Our educational content accurately reflects the histories and experiences of the learners we serve
- Our educational content prompts deeper discussions with learners and motivates them to expand their own learning (and worldview)

While we work hard to present unbiased content, we want to hear from you about any concerns or needs with this Pearson product so that we can

investigate and address them.

Please contact us with concerns about any potential bias at
<https://www.pearson.com/report-bias.html>.

To my mother, who gave me a thirst for knowledge, to my father, who taught me the joys of engineering, and to June, who made it possible to pursue both.

John Canning

For my father Sol Broder, a computer science pioneer, who inspired me to follow in his footsteps.

To my mother Marilyn Broder, for showing me the satisfaction of teaching others.

To Fran, for making my life complete.

Alan Broder

Contents

1. Overview
2. Arrays
3. Simple Sorting
4. Stacks and Queues
5. Linked Lists
6. Recursion
7. Advanced Sorting
8. Binary Trees
9. 2-3-4 Trees and External Storage
10. AVL and Red-Black Trees
11. Hash Tables
12. Spatial Data Structures
13. Heaps
14. Graphs
15. Weighted Graphs
16. What to Use and Why

Appendix A. Running the Visualizations

Appendix B. Further Reading

Appendix C. Answers to Questions

Table of Contents

1. Overview

What Are Data Structures and Algorithms?

Overview of Data Structures

Overview of Algorithms

Some Definitions

Programming in Python

Object-Oriented Programming

Summary

Questions

Experiments

2. Arrays

The Array Visualization Tool

Using Python Lists to Implement the Array Class

The Ordered Array Visualization Tool

Python Code for an Ordered Array Class

Algorithms

Sorting Objects

Big O Notation

Why Not Use Arrays for Everything?

Summary

Questions

Experiments

Programming Projects

3. Simple Sorting

How Would You Do It?

- Bubble Sort
- Selection Sort
- Insertion Sort
- Comparing the Simple Sorts
- Summary
- Questions
- Experiments
- Programming Projects

4. Stacks and Queues

- Different Structures for Different Use Cases
- Stacks
- Queues
- Priority Queues
- Parsing Arithmetic Expressions
- Summary
- Questions
- Experiments
- Programming Projects

5. Linked Lists

- Links
- The Linked List Visualization Tool
- A Simple Linked List
- Linked List Efficiency
- Abstract Data Types and Objects
- Ordered Lists
- Doubly Linked Lists
- Circular Lists
- Iterators
- Summary
- Questions
- Experiments
- Programming Projects

5. Recursion

Triangular Numbers

Factorials

Anagrams

A Recursive Binary Search

The Tower of Hanoi

Sorting with mergesort

Eliminating Recursion

Some Interesting Recursive Applications

Summary

Questions

Experiments

Programming Projects

7. Advanced Sorting

Heapsort

Partitioning

Quicksort

Degenerates to $O(N^2)$ Performance

Radix Sort

Insertion Sort

Summary

Questions

Experiments

Programming Projects

8. Binary Trees

Why Use Binary Trees?

Tree Terminology

An Analogy

How Do Binary Search Trees Work?

Finding a Node

Inserting a Node

Traversing the Tree

inding Minimum and Maximum Key Values

Deleting a Node

The Efficiency of Binary Search Trees

Trees Represented as Arrays

Printing Trees

Duplicate Keys

The BinarySearchTreeTester.py Program

The Huffman Code

Summary

Questions

Experiments

Programming Projects

9. 2-3-4 Trees and External Storage

Introduction to 2-3-4 Trees

The Tree234 Visualization Tool

Python Code for a 2-3-4 Tree

Efficiency of 2-3-4 Trees

B+-3 Trees

External Storage

Summary

Questions

Experiments

Programming Projects

10. AVL and Red-Black Trees

Our Approach to the Discussion

Balanced and Unbalanced Trees

AVL Trees

The Efficiency of AVL Trees

Red-Black Trees

Using the Red-Black Tree Visualization Tool

Experimenting with the Visualization Tool

Rotations in Red-Black Trees

Inserting a New Node
Deletion
The Efficiency of Red-Black Trees
2-3-4 Trees and Red-Black Trees
Red-Black Tree Implementation
Summary
Questions
Experiments
Programming Projects

11. Hash Tables

Introduction to Hashing
Open Addressing
Separate Chaining
Hash Functions
Hashing Efficiency
Hashing and External Storage
Summary
Questions
Experiments
Programming Projects

12. Spatial Data Structures

Spatial Data
Computing Distances Between Points
Circles and Bounding Boxes
Searching Spatial Data
Lists of Points
Grids
Quadrees
Theoretical Performance and Optimizations
Practical Considerations
Further Extensions
Summary

Questions
Experiments
Programming Projects

13. Heaps

Introduction to Heaps
The Heap Visualization Tool
Python Code for Heaps
A Tree-Based Heap
Heapsort
Order Statistics
Summary
Questions
Experiments
Programming Projects

14. Graphs

Introduction to Graphs
Traversal and Search
Minimum Spanning Trees
Topological Sorting
Connectivity in Directed Graphs
Summary
Questions
Experiments
Programming Projects

15. Weighted Graphs

Minimum Spanning Tree with Weighted Graphs
The Shortest-Path Problem
The All-Pairs Shortest-Path Problem
Efficiency
Intractable Problems
Summary

Questions

Experiments

Programming Projects

16. What to Use and Why

Analyzing the Problem

Foundational Data Structures

Special-Ordering Data Structures

Sorting

Specialty Data Structures

External Storage

Onward

Appendix A. Running the Visualizations

For Developers: Running and Changing the Visualizations

For Managers: Downloading and Running the Visualizations

For Others: Viewing the Visualizations on the Internet

Using the Visualizations

Appendix B. Further Reading

Data Structures and Algorithms

Object-Oriented Programming Languages

Object-Oriented Design (OOD) and Software Engineering

Appendix C. Answers to Questions

Chapter 1, “Overview”

Chapter 2, “Arrays”

Chapter 3, “Simple Sorting”

Chapter 4, “Stacks and Queues”

Chapter 5, “Linked Lists”

Chapter 6, “Recursion”

Chapter 7, “Advanced Sorting”

Chapter 8, “Binary Trees”

Chapter 9, “2-3-4 Trees and External Storage”

Chapter 10, “AVL and Red-Black Trees”

Chapter 11, “Hash Tables”

Chapter 12, “Spatial Data Structures”

Chapter 13, “Heaps”

Chapter 14, “Graphs”

Chapter 15, “Weighted Graphs”

Register your copy of *Data Structures & Algorithms in Python* at informit.com for convenient access to downloads, updates, and corrections as they become available. To start the registration process, go to informit.com/register and log in or create an account. Enter the product ISBN 9780134855684 and click Submit. Once the process is complete, you will find any available bonus content under “Registered Products.”

Acknowledgments

From John Canning and Alan Broder

Robert Lafore's Java-based version of this book has been a mainstay in Data Structures courses and professionals' reference shelves around the world for many years. When Alan's Data Structures course at Stern College for Women of Yeshiva University moved on to Python, the inability to use Lafore's book in the course was a real loss. We're thus especially happy to bring this new and revised edition to the world of Python programmers and students.

We'd like to thank the many students at Stern who contributed to this book either directly or indirectly over the past several years. Initial Python versions of Lafore's Java implementations were central to Alan's Python-based courses, and Stern student feedback helped improve the code's clarity, enhanced its performance, and sometimes even identified and fixed bugs!

For their valuable feedback and recommendations on early drafts of this new edition, we are grateful to many students in Alan's Data Structures courses, including Estee Brooks, Adina Bruce, Julia Chase, Hanna Fischer, Limor Kohanim, Elisheva Kohn, Shira Orlan, Shira Pahmer, Jennie Peled, Alexandra Roffe, Avigail Royzenberg, Batia Segal, Penina Waghalter, and Esther Werblowsky. Our apologies if we've omitted anyone's name.

An open-source package of data structure visualizations is available to enhance your study of this book, and Stern students played an active role in the development of the visualization software. John and Alan extend many thanks to the Stern student pioneers and leaders of this project, including Ilana Radinsky, Elana Apfelbaum, Ayliana Teitelbaum, and Lily Polonetsky, as well as the following past and present Stern student contributors: Zoe Abboudi, Ayelet Aharon, Lara Amar, Natania Birnbaum, Adina Bruce, Chani Dubin, Sarah Engel, Sarah Graff, Avigayil Helman, Michal

Kaufman, Sarina Kofman, Rachel Leiser, Talia Leitner, Shani Lewis, Rina Melincoff, Atara Neugroschl, Shira Pahmer, Miriam Rabinovich, Etta Rapp, Shira Sassoon, Mazal Schoenwald, Shira Schneider, Shira Smith, Riva Tropp, Alexandra Volchek, and Esther Werblowsky. Our apologies if we have left anyone off this list.

Many thanks go to Professor David Matuszek of the University of Pennsylvania for his early contributions of ideas and PowerPoint slides when Alan first started teaching Data Structures at Stern. Many of the slides available in the Instructors Resources section have their origin in his clear and well-designed slides. Also, we are grateful to Professor Marian Gidea of the Department of Mathematics of Yeshiva University for his insights into spherical trigonometry.

Finally, we owe a great debt to the talented editors at Pearson who made this book a reality: Mark Taber, Kim Spenceley, Mandie Frank, and Chris Zahn. Without their many talents and patient help, this project would just be an odd collection of text files, drawings, and source code.

From Robert Lafore for the Java-based versions of the book

Acknowledgments to the First Edition

My gratitude for the following people (and many others) cannot be fully expressed in this short acknowledgment. As always, Mitch Waite had the Java thing figured out before anyone else. He also let me bounce the applets off him until they did the job, and extracted the overall form of the project from a miasma of speculation. My editor, Kurt Stephan, found great reviewers, made sure everyone was on the same page, kept the ball rolling, and gently but firmly ensured that I did what I was supposed to do. Harry Henderson provided a skilled appraisal of the first draft, along with many valuable suggestions. Richard S. Wright, Jr., as technical editor, corrected numerous problems with his keen eye for detail. Jaime Niño, Ph.D., of the University of New Orleans, attempted to save me from myself and occasionally succeeded, but should bear no responsibility for my approach or coding details. Susan Walton has been a staunch and much-appreciated supporter in helping to convey the essence of the project to the nontechnical. Carmela Carvajal was invaluable in extending our contacts

with the academic world. Dan Scherf not only put the CD-ROM together, but was tireless in keeping me up to date on rapidly evolving software changes. Finally, Cecile Kaufman ably shepherded the book through its transition from the editing to the production process.

Acknowledgments to the Second Edition

My thanks to the following people at Sams Publishing for their competence, effort, and patience in the development of this second edition. Acquisitions Editor Carol Ackerman and Development Editor Songlin Qiu ably guided this edition through the complex production process. Project Editor Matt Purcell corrected a semi-infinite number of grammatical errors and made sure everything made sense. Tech Editor Mike Kopak reviewed the programs and saved me from several problems. Last but not least, Dan Scherf, an old friend from a previous era, provides skilled management of my code and applets on the Sams website.

About the Author

Dr. John Canning is an engineer, computer scientist, and researcher. He earned an S.B. degree in electrical engineering from the Massachusetts Institute of Technology and a Ph.D. in Computer Science from the University of Maryland at College Park. His varied professions include being a professor of computer science, a researcher and software engineer in industry, and a company vice president. He now is president of Shakumant Software.

Alan Broder is clinical professor and chair of the Department of Computer Science at Stern College for Women of Yeshiva University in New York City. He teaches introductory and advanced courses in Python programming, data structures, and data science. Before joining Stern College, he was a software engineer, designing and building large-scale data analysis systems. He founded and led White Oak Technologies, Inc. as its CEO, and later served as the chairman and fellow of its successor company, Novetta, in Fairfax, Virginia.

Introduction

What's in this book? This book is designed to be a practical introduction to data structures and algorithms for students who have just begun to write computer programs. This introduction will tell you more about the book, how it is organized, what experience we expect readers will have before starting the book, and what knowledge you will get by reading it and doing the exercises.

Who This Book Is For

Data structures and algorithms are the core of computer science. If you've ever wanted to understand what computers can do, how they do it, and what they can't do, then you need a deep understanding of both (it's probably better to say "what computers have difficulty doing" instead of what they can't do). This book may be used as a text in a data structures and/or algorithms course, frequently taught in the second year of a university computer science curriculum. The text, however, is also designed for professional programmers, for high school students, and for anyone else who needs to take the next step up from merely knowing a programming language. Because it's easy to understand, it is also appropriate as a supplemental text to a more formal course. It is loaded with examples, exercises, and supplemental materials, so it can be used for self-study outside of a classroom setting.

Our approach in writing this book is to make it easy for readers to understand how data structures operate and how to apply them in practice. That's different from some other texts that emphasize the mathematical theory, or how those structures are implemented in a particular language or software library. We've selected examples with real-world applications and avoid using math-only or obscure examples. We use figures and visualization programs to help communicate key ideas. We still cover the

complexity of the algorithms and the math needed to show how complexity impacts performance.

What You Need to Know Before You Read This Book

The prerequisites for using this book are: knowledge of some programming language and some mathematics. Although the sample code is written in Python, you don't need to know Python to follow what's happening. Python is not hard to understand, if you've done some procedural and/or object-oriented programming. We've kept the syntax in the examples as general as possible,

More specifically, we use Python version 3 syntax. This version differs somewhat from Python 2, but not greatly. Python is a rich language with many built-in data types and libraries that extend its capabilities. Our examples, however, use the more basic constructs for two reasons: it makes them easier to understand for programmers familiar with other languages, and it illustrates the details of the data structures more explicitly. In later chapters, we do make use of some Python features not found in other languages such as generators and list comprehensions. We explain what these are and how they benefit the programmer.

Of course, it will help if you're already familiar with Python (version 2 or 3). Perhaps you've used some of Python's many data structures and are curious about how they are implemented. We review Python syntax in [Chapter 1, "Overview,"](#) for those who need an introduction or refresher. If you've programmed in languages like Java, C++, C#, JavaScript, or Perl, many of the constructs should be familiar. If you've only programmed using functional or domain-specific languages, you may need to spend more time becoming familiar with basic elements of Python. Beyond this text, there are many resources available for novice Python programmers, including many tutorials on the Internet.

Besides a programming language, what should every programmer know? A good knowledge of math from arithmetic through algebra is essential. Computer programming is symbol manipulation. Just like algebra, there are ways of transforming expressions to rearrange terms, put them in different

forms, and make certain parts more prominent, all while preserving the same meaning. It's also critical to understand exponentials in math. Much of computer science is based on knowing what raising one number to a power of another means. Beyond math, a good sense of organization is also beneficial for all programming. Knowing how to organize items in different ways (by time, by function, by size, by complexity, and so on) is crucial to making programs efficient and maintainable. When we talk about efficiency and maintainability, they have particular meanings in computer science.

Efficiency is mostly about how much time it takes to compute things but can also be about the amount of space it takes. *Maintainability* refers to the ease of understanding and modifying your programs by other programmers as well as yourself.

You'll also need knowledge of how to find things on the Internet, download and install software, and run them on a computer. The instructions for downloading and running the visualization programs can be found in [Appendix A](#) of this book. The Internet has made it very easy to access a cornucopia of tools, including tools for learning programming and computer science. We expect readers to already know how to find useful resources and avoid sources that might provide malicious software.

What You Can Learn from This Book

As you might expect from its title, this book can teach you about how data structures make programs (and programmers) more efficient in their work. You can learn how data organization and its coupling with appropriate algorithms greatly affect what can be computed with a given amount of computing resources. This book can give you a thorough understanding of how to implement the data structures, and that should enable you to implement them in any programming language. You can learn the process of deciding what data structure(s) and algorithms are the most appropriate to meet a particular programming request. Perhaps most importantly, you can learn when an algorithm and/or data structure will *fail* in a given use case. Understanding data structures and algorithms is the core of computer science, which is different from being a Python (or other language) programmer.

The book teaches the fundamental data structures that every programmer should know. Readers should understand that there are many more. These basic data structures work in a wide variety of situations. With the skills you develop in this book, you should be able to read a description of another data structure or algorithm and begin to analyze whether or not it will outperform or perform worse than the ones you've already learned in particular use cases.

This book explains some Python syntax and structure, but it will not teach you all its capabilities. The book uses a subset of Python's full capabilities to illustrate how more complex data structures are built from the simpler constructs. It is not designed to teach the basics of programming to someone who has never programmed. Python is a very high-level language with many built-in data structures. Using some of the more primitive types such as arrays of integers or record structures, as you might find in C or C++, is somewhat more difficult in Python. Because the book's focus is the implementation and analysis of data structures, our examples use approximations to these primitive types. Some Python programmers may find these examples unnecessarily complex, knowing about the more elegant constructs provided with the language in standard libraries. If you want to understand computer science, and in particular, the complexity of algorithms, you must understand the underlying operations on the primitives. When you use a data structure provided in a programming language or from one of its add-on modules, you will often have to know its complexity to know whether it will work well for your use case. Understanding the core data structures, their complexities, and trade-offs will help you understand the ones built on top of them.

All the data structures are developed using ***object-oriented programming*** (OOP). If that's a new concept for you, the review in [Chapter 1](#) of how classes are defined and used in Python provides a basic introduction to OOP. You should not expect to learn the full power and benefits of OOP from this text. Instead, you will learn to implement each data structure as a ***class***. These classes are the *types of objects* in OOP and make it easier to develop software that can be reused by many different applications in a reliable way.

The book uses many examples, but this is not a book about a particular application area of computer science such as databases, user interfaces, or

artificial intelligence. The examples are chosen to illustrate typical applications of programs, but all programs are written in a particular context, and that changes over time. A database program written in 1970 may have appeared very advanced at that time, but it might seem very trivial today. The examples presented in this text are designed to teach how data structures are implemented, how they perform, and how to compare them when designing a new program. The examples should not be taken as the most comprehensive or best implementation possible of each data structure, nor as a thorough review of all the potential data structures that could be appropriate for a particular application area.

Structure

Each chapter presents a particular group of data structures and associated algorithms. At the end of the chapters, we provide review questions covering the key points in the chapter and sometimes relationships to previous chapters. The answers for these can be found in [Appendix C](#), “[Answers to Questions](#).” These questions are intended as a self-test for readers, to ensure you understood all the material.

Many chapters suggest *experiments* for readers to try. These can be individual thought experiments, team assignments, or exercises with the software tools provided with the book. These are designed to apply the knowledge just learned to some other area and help deepen your understanding.

Programming projects are longer, more challenging programming exercises. We provide a range of projects of different levels of difficulty. These projects might be used in classroom settings as homework assignments. Sample solutions to the programming projects are available to qualified instructors from the publisher.

History

Mitchell Waite and Robert Lafore developed the first version of this book and titled it *Data Structures and Algorithms in Java*. The first edition was published in 1998, and the second edition, by Robert, came out in 2002.

John Canning and Alan Broder developed this version using Python due to its popularity in education and commercial and noncommercial software development. Java is widely used and an important language for computer scientists to know. With many schools adopting Python as a first programming language, the need for textbooks that introduce new concepts in an already familiar language drove the development of this book. We expanded the coverage of data structures and updated many of the examples.

We've tried to make the learning process as painless as possible. We hope this text makes the core, and frankly, the beauty of computer science accessible to all. Beyond just understanding, we hope you find learning these ideas fun. Enjoy yourself!

1. Overview

You have written some programs and learned enough to think that programming is fun, or at least interesting. Some parts are easy, and some parts are hard. You'd like to know more about how to make the process easier, get past the hard parts, and conquer more complex tasks. You are starting to study the heart of computer science, and that brings up many questions. This chapter sets the stage for learning how to make programs that work properly and fast. It explains a bunch of new terms and fills in background about the programming language that we use in the examples.

In This Chapter

- [What Are Data Structures and Algorithms?](#)
- [Overview of Data Structures](#)
- [Overview of Algorithms](#)
- [Some Definitions](#)
- [Programming in Python](#)
- [Object-Oriented Programming](#)

What Are Data Structures and Algorithms?

Data organizations are ways data is arranged in the computer using its various storage media (such as random-access memory, or RAM, and disk) and how that data is interpreted to represent something. **Algorithms** are the procedures used to manipulate the data in these structures. The way data is arranged can simplify the algorithms and make algorithms run faster or slower. Together, the data organization and the algorithm form a **data structure**. The data structures

act like building blocks, with more complex data structures using other data structures as components with appropriate algorithms.

Does the way data is arranged and the algorithm used to manipulate it make a difference? The answer is a definite yes. From the perspective of nonprogrammers, it often seems as though computers can do anything and do it very fast. That's not really true. To see why, let's look at a nonprogramming example.

When you cook a meal, a collection of ingredients needs to be combined and manipulated in specific ways. There is a huge variety of ways that you could go about the individual steps needed to complete the meal. Some of those methods are going to be more efficient than others. Let's assume that you have a written recipe, but are working in an unfamiliar kitchen, perhaps while visiting a friend. One method for making the meal would be *Method A*:

1. Read through the full recipe noting all the ingredients it mentions, their quantities, and any equipment needed to process them.
2. Find the ingredients, measure out the quantity needed, and store them.
3. Get out all the equipment needed to complete the steps in the recipe.
4. Go through the steps of the recipe in the order specified.

Let's compare that to *Method B*:

1. Read the recipe until you identify the first set of ingredients or equipment that is needed to complete the first step.
2. Find the identified ingredients or equipment.
3. Measure any ingredients found.
4. Perform the first step with the equipment and ingredients already found.
5. Return to the beginning of this method and repeat the instructions replacing the word *first* with *next*. If there is no next step, then quit.

Both methods are *complete* in that they should finish the complete recipe if all the ingredients and equipment are available. For simple recipes, they should take about the same amount of time too. The methods differ as the recipes get more complex. For example, what if you can't find the fifth ingredient? In method A, that issue is identified at the beginning before any other ingredients are combined. While neither method really explains what to do about

exceptions like a missing ingredient, you can still compare them under the assumption that you handle the exceptions the same way.

A missing ingredient could be handled in several ways: find a substitute ingredient, broaden the search for the ingredient (look in other rooms, ask a neighbor, go to the market), or ignore the ingredient (an optional garnish). Each of those remedies takes some time. If there is one missing ingredient and two cooks using the different methods handle it in the same way, both are delayed the same amount of time. If there are multiple missing ingredients, however, Method A should identify those earlier and allow for the possibility of getting all the missing ingredients in one visit to a neighbor or a market. The time savings of combining the tasks of replacing missing ingredients could be significant (imagine the market being far away or neighbors who want to talk for hours on every visit).

The order of performing the operations could have significant impact on the time needed to complete the meal. Another difference could be in the quality of the meal. For example, in Method B the cook would perform the first “step” and then move on to the next step. Let’s assume those use two different groups of ingredients or equipment. If finding or measuring the ingredients, or getting the equipment for the later steps takes significant time, then the results of the first step sit around for a significant time. That can have a bad effect on the quality of the meal. The cook might be able to overcome that effect in some circumstances by, say, putting the results of the first step in a freezer or refrigerator and then bringing them back to room temperature later. The cook would be preserving the quality of the food at the expense of the time needed to prepare it.

Would Method B ever be desirable if it takes longer or risks degrading the quality of the food? Perhaps. Imagine that the cook is preparing this meal in the unfamiliar kitchen of a family relative. The kitchen is full of family members, and each one is trying to make part of the meal. In this crowded situation, it could be difficult for each cook to get out all of their ingredients and equipment at once. There might not be enough counter space, or mixing bowls, or knives, for example, for each cook to have all their items assembled at the beginning. The cooks could be constrained to work on individual steps while waiting for equipment, space, or ingredients to become available. In this case, Method B could have advantages over asking all the cooks to work one at a time using Method A.

Coming back to programming, the algorithm specifies the sequence of operations that are to be performed, much like the steps in the recipe. The data organizations are somewhat analogous to how the ingredients are stored, laid out in the kitchen, and their proximity to other ingredients and equipment. For example, having the ingredient in the kitchen makes the process much faster than if the ingredient needs to be retrieved from a neighbor or the market. You can think of the amount of space taken up by spreading out the ingredients in various locations as the amount of space needed for the algorithm. Even if all the ingredients are in the kitchen where the cook is, there are ways of setting up the ingredients to make the cooking tasks go faster. Having them compactly arranged in the order they are needed minimizes the amount of moving around the cook must do. The organization of the ingredients can also help if a cook must be replaced by another cook in the middle of the preparation; understanding where each of the ingredients fits in to the recipe is faster if the layout is organized. This is another reason why good data organization is important. It also reinforces that concept that the algorithm and the data organization work together to make the data structure.

Data structures are important not just for speed but also to properly model the meaning of the data. Let's say there's an event that many people want to attend and they need to submit their phone number to have a chance to get tickets. Each person can request multiple tickets. If there are fewer tickets than the number of people who want to get them, some method needs to be applied to decide which phone numbers to contact first and determine the number of tickets they will receive. One method would be to go through the phone numbers one at a time and total up the number of tickets until all are given out, then go through the remaining numbers to let them know the tickets are gone. That might be a fair method if the numbers were put in an ordered list defined in a way that potential recipients understood—for example, a chronological list of phone numbers submitted by people interested in the tickets. If the tickets are to be awarded as a sort of lottery, then going through them sequentially means following any bias that is implicit in the order of the list. Randomly choosing a number from the list, contacting the buyer, and then removing the number would be fairer in a lottery system.

Data structures *model systems by assigning specific meanings to each of the pieces and how they interact*. The “systems” are real-world things like first-come, first-served ticket sales, or a lottery giveaway, or how roads connect cities. For the list of phone numbers with ticket requests, a first-come, first-served system needs the list in chronological order, some pointer to where in

the list the next number should be taken, and a pointer to where any newly arriving number should be added (after all previous list entries). The lottery system would need a different organization, and modeling the map of roads and cities needs another. In this book we examine a lot of different data structures. Each one has its strengths and weaknesses and is applicable to different kinds of real-world problems. It's important to understand how each one operates, whether it correctly models the behavior needed by a particular problem area, whether it will operate efficiently to perform the operations, and whether it can "scale" well. We say a data structure or algorithm scales well if it will perform as efficiently as possible as the amount of data grows.

Overview of Data Structures

As we've discussed, not every data structure models every type of problem. Or perhaps a better way to put it is that the structures model the problem awkwardly or inefficiently. You can generalize the data structures somewhat by looking at the common operations that you are likely to do across all of them. For example, to manage the requests for tickets, you need to

- **Add** a new phone number (for someone who wants one or more tickets)
- **Remove** a phone number (for someone who later decides they don't want tickets)
- **Find** a particular phone number (the next one to get a ticket by some method, or to look up one by its characteristics)
- **List** all the phone numbers (show all the phone numbers exactly once, that is, without repeats except, perhaps, for cases where multiple identical entries were made)

These four operations are needed for almost every data structure that manages a large collection of similar items. We call them **insertion**, **deletion**, **search**, and **traversal**.

Here's a list of the data structures covered in this book and some of their advantages and disadvantages with respect to the four operations. [Table 1-1](#) shows a very high-level view of the structures; we look at them in much more detail in the following chapters. One aspect mentioned in this table is the data structure's **complexity**. In this context, we're referring to the structure's ability

to be understood easily by programmers, not how quickly it can be manipulated by the computer.

Table 1-1 *Comparison of Different Data Types*

Data Structure	Advantages	Disadvantages
Array	Simple. Fast insertion as long as index is known. Fast traversal.	Slow search, size must be known at beginning. Slow to grow.
Ordered array	Same as array but quicker search for items if looking by sort key.	Same as array.
Stack	Simple. Fast for last-in, first-out (LIFO) insertion and deletion. Fast traversal.	Slow search for items other than last-in, first-out.
Queue	Moderately complex. Fast for first-in, first-out (FIFO) insertion and deletion. Fast traversal.	Slow search for items other than first-in, first-out.
Linked List	Moderately complex. Fast insertion and deletion for a known position in list. Easy to grow and shrink. Fast traversal.	Slow search.
Binary tree	Moderately complex. Quick search, insertion, and deletion when tree is balanced. Easy to grow and shrink. Fast traversal.	Deletion algorithm is complex and balancing can be time-consuming.
Red-Black tree	Quick search, insertion, and deletion. Tree always balanced. Fast traversal.	Complex.
2-3-4 tree	Quick search, insertion, and deletion. Tree always balanced. Similar trees are good for disk storage. Easy to grow and shrink. Fast traversal.	Complex.
Quadtree	Quick search, insertion, and deletion by 2 dimensional coordinates. Easy to grow and shrink. Fast traversal.	Complex.
Hash table	Fast search. Fast insertion and quick deletion in most cases.	Complex. Can take more space than other data structures. Traversal may be slightly slower than others.
Heap	Moderately complex. Quick insertion and quick deletion of items in sorted order. Fast traversal.	Slow search for items other than minimum or maximum.
Graph	Fast insertion and unordered traversal of nodes and edges	Complex. Slow search (by label) and deletion. Traversal can be slow depending on path constraints. Can take more space than other data structures.

Overview of Algorithms

Algorithms are ways to implement an operation using a data structure or group of structures. A single algorithm can sometimes be applied to multiple data structures with each data structure needing some variations in the algorithm. For example, a depth first search algorithm applies to all the tree data structures, perhaps the graph, and maybe even stacks and queues (consider a stack as a tree with branching factor of 1). In most cases, however, algorithms are intimately tied to particular data structures and don't generalize easily to others. For example, the way to insert new items or search for the presence of an item is very specific to each data structure. We examine the algorithms for insertion, search, deletion, and traversal for all the data structures. That illustrates how much they vary by data structure and the complexity involved in those structures.

Another core algorithm is **sorting**, where a collection of items is put in a particular order. Ordering the items makes searching for items faster. There are many ways to perform sort operations, and we devote [Chapter 3](#), “[Simple Sorting](#),” to this topic, and revisit the problem in [Chapter 7](#), “[Advanced Sorting](#).”

Algorithms are often defined **recursively**, where part of the algorithm refers to executing the algorithm again on some subset of the data. This very important concept can simplify the definition of algorithms and make it very easy to prove the correctness of an implementation. We study that topic in more detail in [Chapter 6](#), “[Recursion](#).”

Some Definitions

This section provides some definitions of key terms.

Database

We use the term **database** to refer to the complete collection of data that's being processed in a particular situation. Using the example of people interested in tickets, the database could contain the phone numbers, the names, the desired number of tickets, and the tickets awarded. This is a broader definition than what's meant by a relational database or object-oriented database.

Record

Records group related data and are the units into which a database is divided. They provide a format for storing information. In the ticket distribution example, a record could contain a person's name, a person's phone number, a desired number of tickets, and a number of awarded tickets. A record typically includes all the information about some entity, in a situation in which there are many such entities. A record might correspond to a user of a banking application, a car part in an auto supply inventory, or a stored video in a collection of videos.

Field

Records are usually divided into several **fields**. Each field holds a particular kind of data. In the ticket distribution example, the fields could be as shown in [Figure 1-1](#).

Name	
Phone number	
Desired tickets	
Awarded tickets	

Figure 1-1 *A record definition for ticket distribution*

The fields are named and have values. [Figure 1-1](#) shows an empty box representing the storage space for the value. In many systems, the type of value is restricted to a single or small range of data types, just like variables are in many programming languages. For example, the desired number of tickets could be restricted to only integers, or only non-negative integers. In object-oriented systems, objects often represent records, and each object's **attributes** are the fields of that record. The terminology can be different in the various programming languages. Object attributes might be called members, fields, or variables.

Key

When searching for records or sorting them, one of the fields is called the **key** (or **search key** or **sort key**). Search algorithms look for an exact match of the

key value to some target value and return the record containing it. The program calling the search routine can then access all the fields in the record. For example, in the ticket distribution system, you might search for a record by a particular phone number and then look at the number of desired tickets in that record. Another kind of search could use a different key. For example, you could search for a record using the desired tickets as search key and look for people who want three tickets. Note in this case that you could define the search to return the first such record it finds or a collection of all records where the desired number of tickets is three.

Databases vs. Data Structures

The collection of records representing a database is going to require a data structure to implement it. Each record within the database may also be considered a data structure with its own data organization and algorithms. This decomposition of the data into smaller and smaller units goes on until you get to primitive data structures like integers, floating-point numbers, characters, and Boolean values. Not all data structures can be considered databases; they must support insertion, search, deletion, and traversal of records to implement a database.

Programming in Python

Python is a programming language that debuted in 1991. It embraces object-oriented programming and introduced syntax that made many common operations very concise and elegant. One of the first things that programmers new to Python notice is that certain whitespace is significant to the meaning of the program. That means that when you edit Python programs, you should use an editor that recognizes its syntax and helps you create the program as you intend it to work. Many editors do this, and even editors that don't recognize the syntax by filename extension or the first few lines of text can often be configured to use Python syntax for a particular file.

Interpreter

Python is an **interpreted language**, which means that even though there is a compiler, you can execute programs and individual expressions and statements by passing the text to an interpreter program. The compiler works by

translating the source code of a program into **bytecode** that is more easily read by the machine and more efficient to process. Many Python programmers never have to think about the compiler because the Python interpreter runs it automatically, when appropriate.

Interpreted languages have the great benefit of allowing you to try out parts of your code using an interactive command-line interpreter. There are often multiple ways to start a Python interpreter, depending on how Python was installed on the computer. If you use an Integrated Development Environment (IDE) such as IDLE, which comes with most Python distributions, there is a window that runs the command-line interpreter. The method for starting the interpreter differs between IDEs. When IDLE is launched, it automatically starts the command-line interpreter and calls it the *Shell*.

On computers that don't have a Python IDE installed, you can still launch the Python interpreter from a command-line interface (sometimes called a terminal window, or shell, or console). In that command-line interface, type `python` and then press the Return or Enter key. It should display the version of Python you are using along with some other information, and then wait for you to type some expression in Python. After reading the expression, the interpreter decides if it's complete, and if it is, computes the value of the expression and prints it. The example in [Listing 1-1](#) shows using the Python interpreter to compute some math results.

Listing 1-1 *Using the Python Interpreter to Do Math*

```
$ python  
Python 3.6.0 (default, Dec 23 2016, 13:19:00)  
Type "help", "copyright", "credits" or "license" for more information.  
>>> 2019 - 1991  
28  
>>> 2**32 - 1  
4294967295  
>>> 10**27 + 1  
100000000000000000000000000000000000000000000000001  
>>> 10**27 + 1.001  
1e+27  
>>>
```

In [Listing 1-1](#), we've colored the text that you type in *blue italics*. The first dollar sign (\$) is the prompt from command-line interpreter. The Python